

DBMS LAB – TASK 8 – CSE 3

TCL AND DCL Using the **taxation_db**

1. Aim

To understand and apply **Transaction Control Language (TCL)** for controlling tax-related transactions and **Data Control Language (DCL)** for managing user access and privileges in the Taxation Database.

2. Objectives

- Understand transactions in a taxation database.
 - Apply **START TRANSACTION**, **COMMIT**, **ROLLBACK** and **SAVEPOINT**.
 - Perform transactional **INSERT**, **UPDATE** and **DELETE** operations.
 - Create database users and manage their privileges using **GRANT** and **REVOKE**.
 - Verify assigned privileges using **SHOW GRANTS**.
 - Test privileges using a separate database-user connection.
 - Implement the principle of **least privilege** in a taxation system.
-

3. Prerequisites

Students should know:

- **SELECT**, **INSERT**, **UPDATE**, **DELETE**
 - **WHERE** clause
 - Basic joins
 - Tables and relationships
 - Views
 - Primary key and foreign key concepts
 - Basic MySQL user and privilege concepts
-

4. Important Rules and Settings

4.1 Select the database

```
USE taxation_db;
```

Use the Taxation database created in the previous laboratory tasks.

4.2 Disable AUTOCOMMIT

Before performing TCL experiments:

```
SET AUTOCOMMIT = 0;
```

```
SELECT @@AUTOCOMMIT;
```

The value should be:

0

After completing the TCL experiments:

```
SET AUTOCOMMIT = 1;
```

4.3 Use DML for TCL experiments

Use:

- INSERT
- UPDATE
- DELETE

for transaction demonstrations.

Avoid using:

- CREATE
- ALTER
- DROP

inside TCL demonstrations because DDL statements may cause implicit commits in MySQL.

4.4 DCL – Use Two Connections

Connection 1 – Administrator

Used for:

CREATE USER
GRANT
REVOKE
SHOW GRANTS

Connection 2 – Test User

Used for testing:

SELECT
INSERT
UPDATE
DELETE

Always test privileges using the account to which the privilege was actually granted.

To verify the current account:

```
SELECT CURRENT_USER();
```

5. TCL – Basic Syntax and Examples

5.1 START TRANSACTION

```
START TRANSACTION;
```

Starts a new transaction.

5.2 COMMIT

Suppose a taxpayer's income record needs to be corrected.

```
START TRANSACTION;
```

```
UPDATE Income_Record  
SET income_amount = 850000  
WHERE income_record_id = 3;
```

```
COMMIT;
```

COMMIT permanently saves the changes made during the transaction.

5.3 ROLLBACK

Suppose an incorrect income amount was entered.

```
START TRANSACTION;
```

```
UPDATE Income_Record  
SET income_amount = 950000  
WHERE income_record_id = 3;
```

```
ROLLBACK;
```

The uncommitted modification is cancelled.

5.4 SAVEPOINT

```
START TRANSACTION;
```

```
UPDATE Income_Record  
SET income_amount = 850000  
WHERE income_record_id = 3;
```

```
SAVEPOINT income_update;
```

```
UPDATE Income_Record  
SET income_amount = 1200000  
WHERE income_record_id = 6;
```

SAVEPOINT creates a point inside the transaction to which we can partially roll back.

5.5 ROLLBACK TO SAVEPOINT

```
ROLLBACK TO SAVEPOINT income_update;
```

```
COMMIT;
```

The changes made **after** the savepoint are removed, while changes made before the savepoint remain.

5.6 RELEASE SAVEPOINT

```
RELEASE SAVEPOINT income_update;
```

Removes the specified savepoint.

6. DCL – Basic Syntax and Examples

6.1 CREATE USER

```
CREATE USER 'tax_clerk1'@'localhost'  
IDENTIFIED BY 'Tax@123';
```

Creates a separate database account.

6.2 GRANT

```
GRANT SELECT  
ON taxation_db.Income_Record  
TO 'tax_clerk1'@'localhost';
```

The user can now retrieve information from [Income_Record](#).

6.3 Multiple Privileges

```
GRANT SELECT, INSERT  
ON taxation_db.Income_Record
```

TO 'tax_clerk1'@'localhost';

The user receives only **SELECT** and **INSERT**.

UPDATE and **DELETE** are not automatically granted.

6.4 Grant SELECT on a View

Suppose a view called **Taxpayer_Income_Summary** exists:

```
GRANT SELECT
ON taxation_db.Taxpayer_Income_Summary
TO 'tax_clerk1'@'localhost';
```

This allows the user to access the view without necessarily giving direct access to all underlying tables.

6.5 REVOKE

```
REVOKE INSERT
ON taxation_db.Income_Record
FROM 'tax_clerk1'@'localhost';
```

Removes the **INSERT** privilege.

6.6 SHOW GRANTS

```
SHOW GRANTS FOR 'tax_clerk1'@'localhost';
```

Displays privileges assigned to the user.

7. Laboratory Activities

Part A – Verify Previous Database

Task 1 Select the Taxation database and display all available tables.

USE taxation_db;

SHOW TABLES;

Verify the tables:

Taxpayer

Income_Category

Financial_Year

Income_Record

Task 2 Display the records from all four tables and verify that the relationships between taxpayers, income categories, financial years and income records are available.

Part B – TCL: Basic Transactions

Task 1 Disable **AUTOCOMMIT**, verify its value, start a transaction, modify the income amount of one taxpayer's income record, and verify the modified record before committing.

Task 2 A taxpayer has corrected their reported annual income.

Update the corresponding **Income_Record**, verify the change, and use **COMMIT** to permanently save the corrected income.

Task 3 An incorrect income amount was entered for a taxpayer.

Modify the income amount, verify the incorrect modification, execute **ROLLBACK**, and confirm that the original income amount has been restored.

Task 4 A new income record was mistakenly entered for a taxpayer.

Insert the record, verify its existence, execute **ROLLBACK**, and confirm that the newly inserted record no longer exists.

Task 5 An income record was accidentally deleted during tax-data maintenance.

Delete the record, verify its absence, execute **ROLLBACK**, and confirm that the original record has been restored.

Task 6 Perform **two different DML operations** in a single transaction—for example, update one income record and insert another income record—and permanently save both operations using **COMMIT**.

Part C – TCL: Savepoints

Task 1 Update the annual income of one taxpayer and create a savepoint.

Then update the income of a second taxpayer.

Use **ROLLBACK TO SAVEPOINT** and verify that:

- the first update remains;
 - the second update is cancelled.
-

Task 2 Insert a new income record for one taxpayer.

Create a savepoint.

Then modify another taxpayer's income record.

Rollback to the savepoint and commit the remaining valid change.

Task 3 Create **two savepoints** during a transaction involving different **Income_Record** entries.

Perform additional modifications and rollback to the **first savepoint**.

Verify which modifications remain.

Task 4 Perform the following operations within one transaction:

1. Insert a new income record.
2. Update an existing income record.
3. Delete another income record.
4. Create a savepoint before the deletion.
5. Rollback to the savepoint.
6. Commit the valid changes.

Verify the final database state.

Task 5 Create a savepoint while modifying an income record.

Release the savepoint using:

RELEASE SAVEPOINT savepoint_name;

Then attempt to rollback to the released savepoint and observe the result.

Task 6 Demonstrate the difference between:

ROLLBACK;

and

ROLLBACK TO SAVEPOINT;

using two different **Income_Record** modifications.

Part D – DCL: Users and Privileges

Before starting, keep **two MySQL connections** available:

- Administrator connection
- Test-user connection

Task 1 Create a MySQL user named:

tax_clerk1

Verify that the user has been created and connect using the new credentials.

Task 2 Grant only **SELECT** privilege on the **Taxpayer** table to **tax_clerk1**.

Verify the privilege using:

```
SHOW GRANTS FOR 'tax_clerk1'@'localhost';
```

Then connect as **tax_clerk1** and test a **SELECT** operation.

Task 3 Grant **INSERT** privilege on **Income_Record** to **tax_clerk1**.

Verify the privilege and test whether the user can insert a new income record.

Task 4 Without granting **UPDATE**, attempt to modify an existing **Income_Record** as **tax_clerk1**.

Verify that the operation is denied.

Task 5 Grant **SELECT** privilege on a suitable taxation summary View to **tax_clerk1**.

Test whether the user can retrieve data from the View.

Task 6 Revoke the **INSERT** privilege from **tax_clerk1**.

Verify the privileges again and reconnect as the test user.

Attempt an **INSERT** operation and verify that it is denied.

Part E – DCL: Security and Least Privilege

Task 1 Create a user named:

tax_data_entry

Give the user only:

SELECT
INSERT

on **Income_Record**.

Test both permitted and restricted operations.

Task 2 Create another user named:

tax_officer

Grant:

SELECT
INSERT
UPDATE

on **Income_Record**.

Attempt a **DELETE** operation and verify whether it is permitted.

Task 3 Grant **SELECT** privilege on a taxation summary View to **tax_officer**.

Verify that the user can retrieve the required summarized information through the View.

Task 4 Grant multiple privileges to a taxation user.

For example:

```
SELECT
INSERT
UPDATE
```

Then revoke only **UPDATE**.

Verify that:

- **SELECT** still works.
 - **INSERT** still works.
 - **UPDATE** is no longer permitted.
-

Task 5 Use:

```
SHOW GRANTS
```

to compare the privileges assigned to **tax_data_entry** and **tax_officer**.

Explain why their privileges are different based on their roles in the taxation system.

Task 6 Implement a **minimum-privilege Tax Data Entry user** who can:

- view taxpayer income information;
- enter new income records;

but cannot:

- modify existing income records;
- delete income records.

Test all four operations using the test-user connection.

Part F – Integrated TCL and DCL

Task 1 Simulate the submission of a new annual income record for a taxpayer.

Use:

START TRANSACTION

↓

INSERT

↓

Verify

↓

COMMIT

Confirm that the record remains permanently in the database.

Task 2 Simulate an incorrect correction to a taxpayer's annual income.

Perform the update inside a transaction and use **ROLLBACK** to restore the original income value.

Task 3 Perform multiple modifications to different taxpayers' income records.

Create a **SAVEPOINT** after the first valid modification.

Make another incorrect modification and use:

ROLLBACK TO SAVEPOINT;

Commit the remaining valid modification.

Task 4 Create a **minimum-privilege Tax Data Entry user**.

Grant only the privileges required to enter and view income records.

Test:

SELECT → permitted
INSERT → permitted
UPDATE → restricted
DELETE → restricted

Task 5 Create a taxation summary View and grant **SELECT** access to a user.

Test the difference between accessing the permitted View and directly accessing a restricted underlying table.

Task 6 Grant an unnecessary privilege to a taxation user.

Then:

1. Verify the privilege using **SHOW GRANTS**.
 2. Revoke the unnecessary privilege.
 3. Verify the final privileges.
 4. Reconnect using the test account.
 5. Test the revoked operation.
-

8. Quick Reference

TCL Progression

START TRANSACTION;



INSERT / UPDATE / DELETE



SAVEPOINT (Optional)



COMMIT / ROLLBACK

Command

Purpose

START TRANSACTION	Begins a transaction
COMMIT	Permanently saves changes
ROLLBACK	Cancels uncommitted changes
SAVEPOINT	Creates a partial rollback point
ROLLBACK TO SAVEPOINT	Removes changes after a savepoint
RELEASE SAVEPOINT	Removes a savepoint
SET AUTOCOMMIT	Controls automatic committing

DCL Progression

Administrator



CREATE USER



GRANT



SHOW GRANTS



Test User



Test Operations



REVOKE



SHOW GRANTS / Test Again

Command	Purpose
CREATE USER	Creates a database account
GRANT	Provides privileges
REVOKE	Removes privileges
SHOW GRANTS	Displays assigned privileges
Least Privilege	Gives only required permissions

9. TCL vs DCL

TCL	DCL
Controls transactions	Controls database access
Works with data changes	Works with users and privileges
COMMIT	GRANT
ROLLBACK	REVOKE
SAVEPOINT	CREATE USER
ROLLBACK TO SAVEPOINT	SHOW GRANTS

10. Real-World Taxation Access Model

Role	Required Access
Taxpayer	View/submit their required income information
Tax Data Entry User	SELECT, INSERT on relevant income records
Tax Officer	SELECT, INSERT, UPDATE as required
Database Administrator	Users, privileges, tables, Views and database security

The central principle is:

Assign database privileges according to the responsibility of the user.

11. Final Verification and Cleanup

```
USE taxation_db;

SHOW TABLES;

SELECT * FROM Taxpayer;

SELECT * FROM Income_Category;

SELECT * FROM Financial_Year;

SELECT * FROM Income_Record;

SELECT @@AUTOCOMMIT;

SET AUTOCOMMIT = 1;

SELECT CURRENT_USER();

Verify the test user's privileges:

SHOW GRANTS FOR 'tax_clerk1'@'localhost';

If the user was created only for the laboratory:

DROP USER IF EXISTS 'tax_clerk1'@'localhost';

Do not remove existing administrative accounts.
```

12. GitHub Submission

StudentID-repo

```
|
└─ Week-8-Lab
    |
    └─ week8_tcl_dcl_taxation.sql
```